

Mini Game Hub Report

Made by AMAN SIHAG AND AKSHIT

27-04-2026

Contents

1	Project Specification	3
2	Overview	3
3	System Architecture	3
4	Authentication System	4
5	Game Engine	4
6	Games Implemented	5
6.1	Tic Tac Toe	5
6.2	Othello	5
6.3	Connect Four	5
7	Screenshots and UI	5
7.1	Game Menu	5
7.2	Tic Tac Toe	6
7.3	Game Over	7
7.4	Leaderboard	8
7.5	Graphs	8
8	Leaderboard and Analytics	8
9	Technical Constraints	9
10	Hurdles Faced	9
11	Future Improvements	9
12	References	10

1 Project Specification

Technologies Used:

- Bash
- Python 3.10
- Pygame
- NumPy
- Matplotlib
- SHA256 algorithm for password hashing
- Sed and awk for data manipulation in Bash

2 Overview

The Mini Game Hub is a secure multiplayer gaming platform where two authenticated users can play multiple games through a graphical interface. The system integrates Bash for authentication using SHA256 algorithm to store the encoded passwords and Python for game execution, data recording, and analytics. The games implemented include Tic Tac Toe, Othello, and Connect Four, each with unique mechanics and win conditions and a common base class. The platform also features a leaderboard and statistical graphs to track player performance and game popularity. The basic game flow is that the users log in, select a game, play against each other, and then the results are recorded for leaderboard and analytics purposes.

3 System Architecture

```
#verbaitem to display the directory structure in a monospaced font
hub/
|-- assets/
|-- materials/
|-- report/
|-- main.sh
|-- game.py
|-- leaderboard.sh
|-- games/
|   |-- tictactoe.py
|   |-- othello.py
|   |-- connect4.py
|-- users.tsv
|-- history.csv
```

Entry point:

```
bash main.sh
```

4 Authentication System

- Two users log in using their username and password.
- Their credentials are verified against the users.tsv file using Bash.
- Then the passwords are hashed using SHA256 algorithm and stored in users.tsv for security.
- New users can register if they don't have an account, and their credentials are added to users.tsv.
- After login, Python game engine is launched , which shows the game menu and allows users to select a game to play.

```
python3 game.py <username1> <username2>
```

5 Game Engine

Game engine is responsible for handling the game logic, user interactions, and data recording. It is implemented in Python using Pygame for the graphical interface and NumPy for board representation and win condition checking. It shows the game menu, allows users to select a game, and then runs the selected game. After the game is over, the game over menu appears, which has options to play again, return to the main menu, or show leaderboard. If the users choose to play again, the same game is restarted. If they choose to return to the main menu, they are taken back to the game selection screen. If they choose to show leaderboard, the leaderboard menu with sorting options appears, which allows users to view the leaderboard based on wins, losses, or win/loss ratio.

- Player handling
- Turn switching
- NumPy-based board
- Win condition checking
- Data recording in history.csv
- GUI for all interactions
- Leaderboard display and sorting
- Statistical graph generation using Matplotlib
- Game over menu with options to play again, return to main menu, or show leaderboard

6 Games Implemented

6.1 Tic Tac Toe

- 10*10 grid
- Win condition: 5 in a row

6.2 Othello

- 8*8 board
- Win condition: Most pieces on the board at the end of the game

6.3 Connect Four

- 7*7 grid
- Win condition: 4 in a row

7 Screenshots and UI

7.1 Game Menu

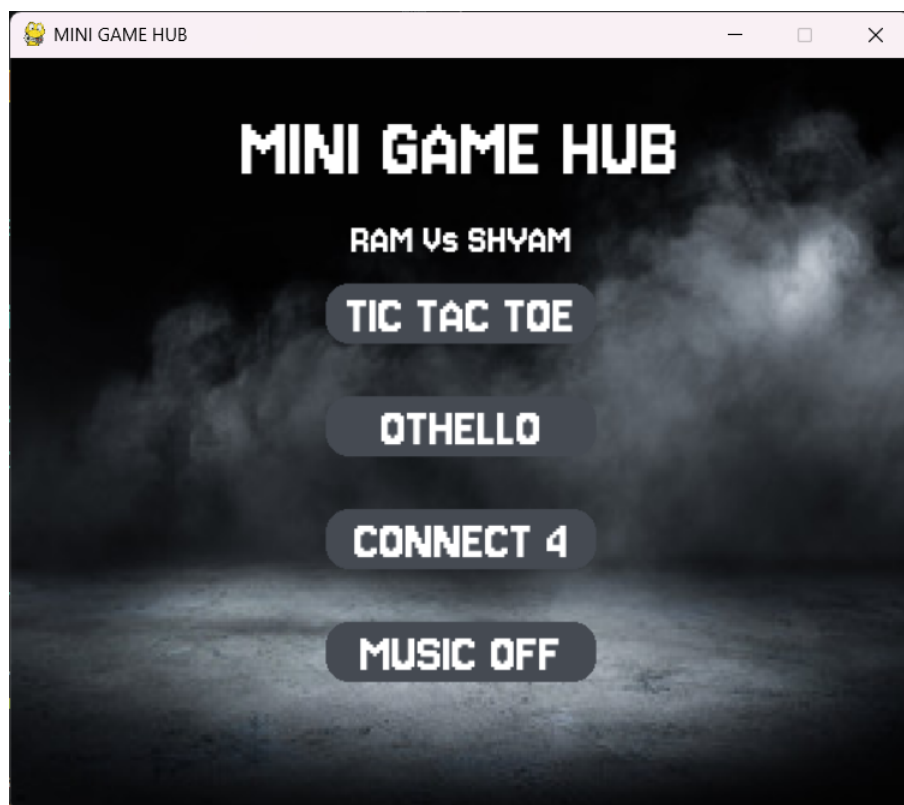


Figure 1: Main Game Menu

7.2 Tic Tac Toe

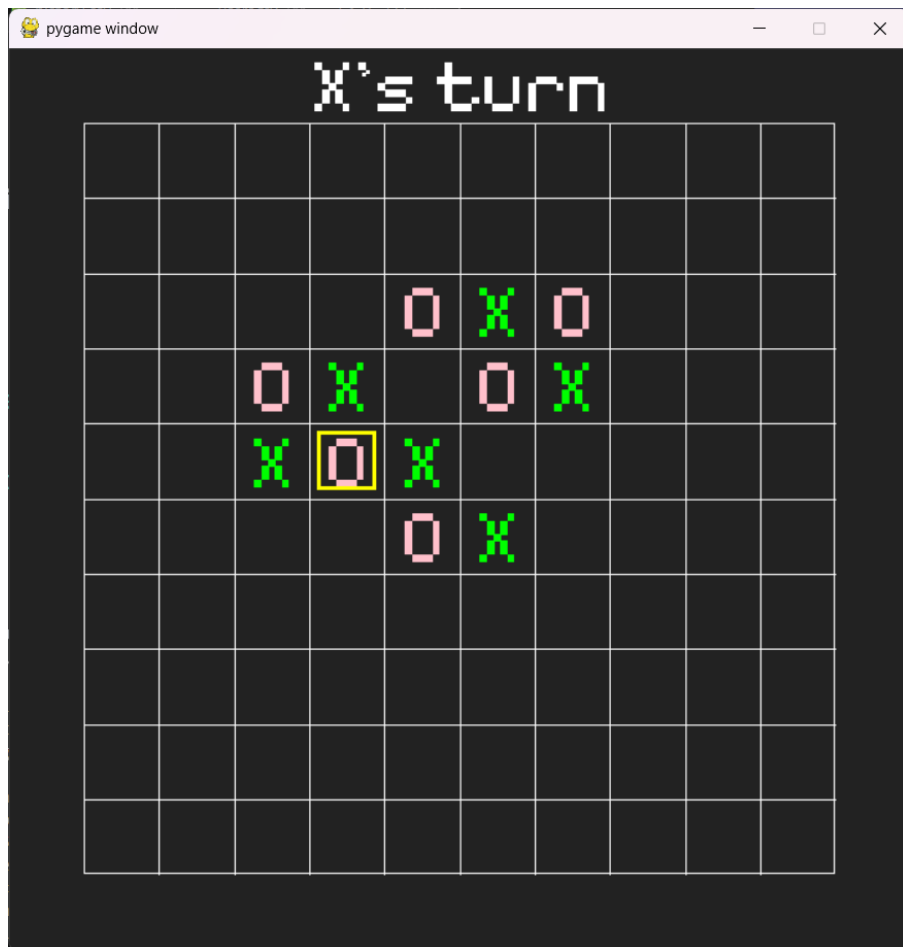


Figure 2: Tic Tac Toe Gameplay

7.3 Game Over

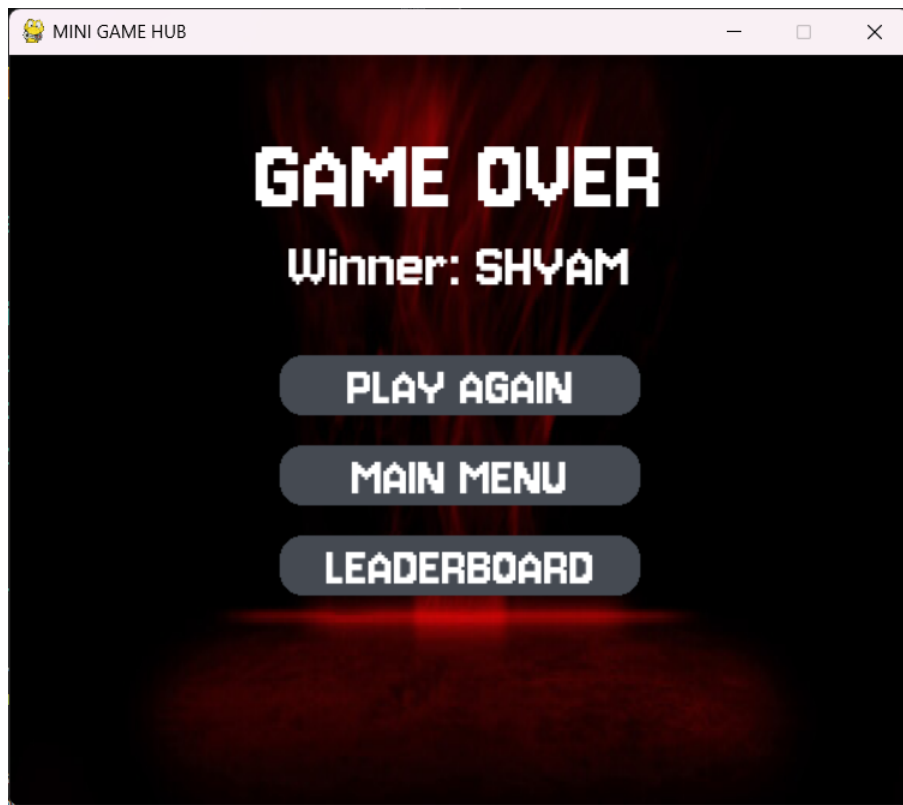


Figure 3: Game Over Screen

7.4 Leaderboard

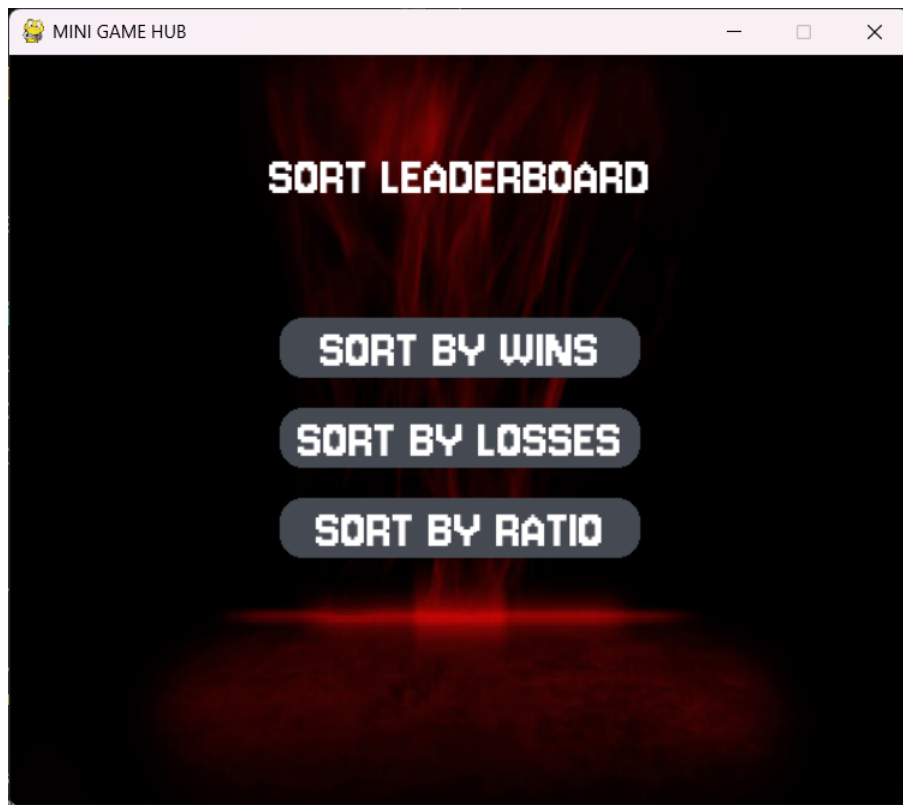


Figure 4: Leaderboard Interface

7.5 Graphs

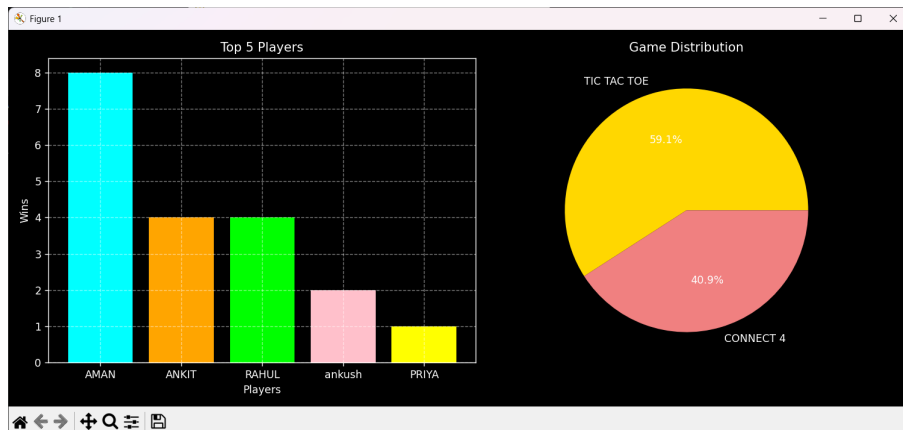


Figure 5: Statistical Graphs

8 Leaderboard and Analytics

- Game results are stored in history.csv
- Leaderboard is generated using Bash
- Displays wins, losses, and ratios for each player

- Sorting options for wins, losses, and win/loss ratio
- Graphs of top 5 players and pie chart showing percentage of games played for each game type using Matplotlib

9 Technical Constraints

- NumPy used for all board representations
- No manual loops for win checking
- GUI for everything using Pygame

10 Hurdles Faced

- Connecting the games to the main game engine game.py
- Implementing the game logic for each game, especially othello
- Integrating Bash with Python
- Writing logic for othello
- Intergrating all the code into classes after making them work.
- Changing all the working functions into numpy methods, and as we don't know if we have a method for this thing.

Solutions:

- Used subprocess for communication
- Just kept coding and then checking
- Used vim macros to add self. in front
- Structured event loops properly
- Read documentation using find feature

11 Future Improvements

- Making it online so that players can play accross internet
- Better UI animations
- Player ranking system

12 References

- <https://pyga.me>
- <https://numpy.org>
- <https://worldothello.org>
- <https://boardgamegeek.com>